

# Massively parallel quantum computer simulator

K. De Raedt<sup>a</sup>, K. Michielsen<sup>b</sup>, H. De Raedt<sup>b,\*</sup>, B. Trieu<sup>c</sup>, G. Arnold<sup>c</sup>, M. Richter<sup>c</sup>,  
Th. Lippert<sup>c</sup>, H. Watanabe<sup>d</sup>, N. Ito<sup>e</sup>

<sup>a</sup> Department of Computer Science, University of Groningen, Blauwborgje 3, NL-9747 AC Groningen, The Netherlands

<sup>b</sup> Department of Applied Physics, Materials Science Centre, University of Groningen, Nijenborgh 4, NL-9747 AG Groningen, The Netherlands

<sup>c</sup> Zentralinstitut für Angewandte Mathematik, Forschungszentrum Jülich, D-52425 Jülich, Germany

<sup>d</sup> Department of Complex Systems Science, Graduate School of Information Science, Nagoya University, Furouchou, Chikusa-ku, Nagoya 464-8601, Japan

<sup>e</sup> Department of Applied Physics, School of Engineering, The University of Tokyo, Hongo 7-3-1, Bunkyo-ku, Tokyo 113-8656, Japan

Received 21 February 2006; accepted 24 August 2006

Available online 13 October 2006

## Abstract

We describe portable software to simulate universal quantum computers on massive parallel computers. We illustrate the use of the simulation software by running various quantum algorithms on different computer architectures, such as a IBM BlueGene/L, a IBM Regatta p690+, a Hitachi SR11000/J1, a Cray X1E, a SGI Altix 3700 and clusters of PCs running Windows XP. We study the performance of the software by simulating quantum computers containing up to 36 qubits, using up to 4096 processors and up to 1 TB of memory. Our results demonstrate that the simulator exhibits nearly ideal scaling as a function of the number of processors and suggest that the simulation software described in this paper may also serve as benchmark for testing high-end parallel computers.

© 2006 Elsevier B.V. All rights reserved.

PACS: 03.67.Lx; 02.70.-c

Keywords: Quantum computation; Computer simulation; High performance computing; Parallelization

## 1. Introduction

In this paper, we describe a Fortran 90 software package to simulate universal quantum computers [1]. The software runs on various computer architectures, ranging from PCs to high-end (vector) parallel machines. The simulator can perform all the quantum operations that are necessary for universal quantum computation. The maximum number of qubits is set by the memory of the machine on which the code runs. We present simulation results for quantum computers containing up to 36 qubits. In view of the fact that the simulation of quantum systems, such as quantum computers, requires computational resources that grow exponentially with the system size, this represents a significant advance beyond the state of the art, which is currently around 32 qubits [2]. An overview of quantum computer simulator software is given in Ref. [3].

The main focus of the present work is on the design of portable, efficient parallel simulation code for a universal quantum computer. A subsequent paper [4] will explore optimization techniques to further improve the performance of the parallel code. In principle, because of this “universality”, this code for the ideal quantum computer can also be used to simulate physical systems, such as quantum spin models or models for physical realizations of quantum computers, by writing the time evolution of the physical system as a sequence of elementary quantum gate operations [1]. However, this approach is bound to be computationally inefficient for all but nontrivial physical systems. Instead, it is much more effective to allow for additional unitary transformations

\* Corresponding author.

E-mail addresses: [h.a.de.raedt@rug.nl](mailto:h.a.de.raedt@rug.nl) (H. De Raedt), [b.trieu@fz-juelich.de](mailto:b.trieu@fz-juelich.de) (B. Trieu), [g.arnold@fz-juelich.de](mailto:g.arnold@fz-juelich.de) (G. Arnold), [m.richter@fz-juelich.de](mailto:m.richter@fz-juelich.de) (M. Richter), [th.lippert@fz-juelich.de](mailto:th.lippert@fz-juelich.de) (Th. Lippert), [hwatanabe@is.nagoya-u.ac.jp](mailto:hwatanabe@is.nagoya-u.ac.jp) (H. Watanabe), [ito@ap.t.u-tokyo.ac.jp](mailto:ito@ap.t.u-tokyo.ac.jp) (N. Ito).  
URL: <http://www.compphys.net> (H. De Raedt).

that implement operations such as the time evolution under a Heisenberg Hamiltonian in an optimal manner [3,5–7]. Such an extension, which does not affect the intrinsic performance of the code, will be dealt with in a future publication.

The paper is organized as follows. In Section 2, we briefly recall some basic elements of quantum computation. In Section 3, we present a scheme to parallelize the software to simulate a quantum computer on a massive parallel computer. We employ the standard Message Passing Interface (MPI) [8] to implement this scheme. Section 4 describes the main features of the simulator software. In Section 5, we discuss a few nontrivial quantum algorithms, such as an adder of two to five registers of qubits and Shor's factoring algorithm, that are used to validate and benchmark the simulation software. Section 6 presents our benchmark results of running the quantum algorithms on various parallel computers. A discussion is given in Section 7.

## 2. Quantum computation

For convenience of the reader, to make the paper self-contained and to explain the terminology, we summarize some basic aspects of quantum computation.

### 2.1. Quantum computer terminology

The state  $|\Phi\rangle$  of a qubit, the elementary storage unit of a quantum computer, is described by a two-dimensional vector of Euclidean length one:

$$|\Phi\rangle = a(0)|0\rangle + a(1)|1\rangle = a_0|0\rangle + a_1|1\rangle, \quad (1)$$

where  $|0\rangle$  and  $|1\rangle$  denote two orthogonal basis vectors of the two-dimensional vector space and  $a_0 \equiv a(0)$  and  $a_1 \equiv a(1)$  are complex numbers such that  $|a_0|^2 + |a_1|^2 = 1$ . The result of inquiring about the state of a single qubit, that is the outcome of a measurement, is either 0 or 1. The frequency of obtaining 0 (1) can be estimated by repeated measurement of the same state of the qubit and is given by  $|a_0|^2$  ( $|a_1|^2$ ) [9–11].

The internal state of a quantum computer with  $L$  qubits is described by a vector, also called the state vector, in a  $D = 2^L$  dimensional space [1]. Adopting the convention of quantum computation literature [1], the state of an  $L$ -qubit quantum computer is represented by

$$\begin{aligned} |\Phi\rangle &= a(0\dots 00)|0\dots 00\rangle + a(0\dots 01)|0\dots 01\rangle + \dots + a(1\dots 10)|1\dots 10\rangle + a(1\dots 11)|1\dots 11\rangle \\ &= a_0|0\rangle + a_1|1\rangle + \dots + a_{2^L-2}|2^L-2\rangle + a_{2^L-1}|2^L-1\rangle, \end{aligned} \quad (2)$$

where in the last line of Eq. (2), the binary representation of the integers  $0, \dots, 2^L-1$  was used to denote  $|0\rangle \equiv |0\dots 00\rangle, \dots, |2^L-1\rangle \equiv |1\dots 11\rangle$  and  $a_0 \equiv a(0\dots 00), \dots, a_{2^L-1} \equiv a(1\dots 11)$ . We normalize the state vector, that is  $\langle\Phi|\Phi\rangle = 1$ , by rescaling the complex-valued amplitudes  $a_i$  according to

$$\sum_{i=0}^{2^L-1} |a_i|^2 = 1. \quad (3)$$

Note that in this notation it is convenient to number the qubits from 0 to  $L-1$ , that is qubit 0 corresponds to the least significant bit of the integer index that runs from zero to  $2^L-1$ .

A quantum algorithm is a sequence of unitary operations on the vector  $|\Phi\rangle$ . It has been shown that an arbitrary unitary operation can be written as a sequence of single qubit operations and the CNOT operation on two qubits [1,12]. Therefore, single-qubit operations and the CNOT operation are sufficient to construct a universal quantum computer [1]. We call these operations elementary unitary transformations. According to quantum theory, after executing a quantum algorithm, the probability for observing the quantum computer in one of its  $2^L$  states is given by the square of the absolute value of the corresponding element of the state vector.

In the quantum computation literature, the convention is to count each elementary, unitary transformation as one operation on a quantum computer [1]. However, carrying out a unitary operation on a conventional computer requires more than one arithmetic operation and it is customary to determine the performance of an algorithm by counting the number of arithmetic operations. Although the difference between these two ways of expressing the performance of an algorithm should be clear from the context, the reader should keep this difference in mind.

### 2.2. Single-qubit operations

Single-qubit operations that are often used in quantum computation are the Hadamard operation  $H$  and rotations  $X$  and  $Y$  of the state vector by  $\pi/2$  about the  $x$ - and  $y$ -axis of the spin-1/2 operator  $\mathbf{S} = (S^x, S^y, S^z)$ , representing the qubit. The Hadamard

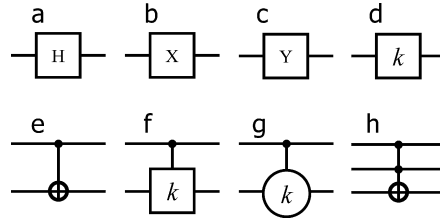


Fig. 1. Graphical representation of some of the basic gates used in quantum computation; (a) Hadamard gate; (b) Rotation by  $\pi/2$  about the  $x$ -axis; (c) Rotation by  $\pi/2$  about the  $y$ -axis; (d) Single qubit phase shift by  $\phi = 2\pi/2^k$ ; (e) CNOT gate; (f) Controlled phase shift by  $\phi = 2\pi/2^k$ ; (g) Controlled  $V$  operation by  $\phi = 2\pi/2^k$ ; (h) Toffoli gate. The horizontal lines denote the qubits involved in the quantum operations. The dots and crosses denote the control and target qubits, respectively.

operation is defined by [1]

$$H|\Phi\rangle = H(a_0|0\rangle + a_1|1\rangle) = \frac{1}{\sqrt{2}}(a_0 + a_1)|0\rangle + \frac{1}{\sqrt{2}}(a_0 - a_1)|1\rangle, \quad (4)$$

the  $X$  operation by

$$X|\Phi\rangle = e^{i\pi S^x/2}(a_0|0\rangle + a_1|1\rangle) = \frac{1}{\sqrt{2}}(a_0 + ia_1)|0\rangle + \frac{1}{\sqrt{2}}(a_1 + ia_0)|1\rangle, \quad (5)$$

and the  $Y$  operation by

$$Y|\Phi\rangle = e^{i\pi S^y/2}(a_0|0\rangle + a_1|1\rangle) = \frac{1}{\sqrt{2}}(a_0 + a_1)|0\rangle + \frac{1}{\sqrt{2}}(a_1 - a_0)|1\rangle. \quad (6)$$

The phase shift operation  $R(\phi)$  is defined by

$$R(\phi)|\Phi\rangle = e^{i\phi/2}e^{-i\phi S^z}(a_0|0\rangle + a_1|1\rangle) = a_0|0\rangle + a_1e^{i\phi}|1\rangle, \quad (7)$$

$R(\phi)$  changes the phase of the amplitude of the  $|1\rangle$  component of the state vector only. Quantum algorithms are often represented by quantum networks, diagrams that show the order of the operations performed on the qubits. The graphical symbols of  $H$ ,  $X$ ,  $Y$ , and  $R(\phi = 2\pi/2^k)$  are shown in Fig. 1. The inverse and transpose of a unitary operation  $U$  are denoted by  $\bar{U}$  and  $U^T$ , respectively.

### 2.3. Two-qubit operations: CNOT and controlled phase shift

Computation requires some form of communication between the qubits. Any form of communication between qubits can be reduced to a combination of single-qubit operations and the CNOT operation, a two-qubit operation [1,12]. By definition, the CNOT gate flips the target qubit if the control qubit is in the state  $|1\rangle$  [1]. If we take qubit zero (that is the least significant bit in the binary notation of an integer) as the control qubit and qubit one as the target qubit then we have

$$\begin{aligned} \text{CNOT}_{10}|\Phi\rangle &= \text{CNOT}_{10}(a_0|00\rangle + a_1|01\rangle + a_2|10\rangle + a_3|11\rangle) \\ &= a_0|00\rangle + a_3|01\rangle + a_2|10\rangle + a_1|11\rangle \\ &= a_0|0\rangle + a_3|1\rangle + a_2|2\rangle + a_1|3\rangle. \end{aligned} \quad (8)$$

The graphical symbol of the CNOT operation is shown in Fig. 1(e). The dot (cross) denotes the control (target) qubit.

Another frequently used operation is the controlled phase shift. The controlled phase shift operation with control qubit 0 and target qubit 1 is defined by

$$\begin{aligned} R_{10}(\phi)|\Phi\rangle &= R_{10}(\phi)(a_0|00\rangle + a_1|01\rangle + a_2|10\rangle + a_3|11\rangle) \\ &= a_0|00\rangle + a_1|01\rangle + a_2|10\rangle + e^{i\phi}a_3|11\rangle. \end{aligned} \quad (9)$$

Graphically, the controlled phase shift  $R_{ji}(\phi = 2\pi/2^k)$  is represented by a vertical line connecting a dot (control qubit) and a box denoting a single qubit phase shift by  $2\pi/2^k$  (see Fig. 1(f)).

The CNOT operation is a special case of the controlled unitary transformation  $V$ . If qubit zero is the control qubit and qubit one is the target qubit then

$$\begin{aligned} V_{10}(\phi)|\Phi\rangle &= V_{10}(\phi)(a_0|00\rangle + a_1|01\rangle + a_2|10\rangle + a_3|11\rangle) \\ &= a_0|00\rangle + \frac{(1 + e^{i\phi})a_1 + (1 - e^{i\phi})a_3}{2}|01\rangle + a_2|10\rangle + \frac{(1 - e^{i\phi})a_1 + (1 + e^{i\phi})a_3}{2}|11\rangle. \end{aligned} \quad (10)$$

The graphical symbol of the controlled  $V$  operation,  $V_{ji}(\phi = 2\pi/2^k)$ , is a vertical line connecting a dot and a circle containing the value  $k$  (see Fig. 1(g)).

## 2.4. Three-qubit operation: Toffoli gate

The Toffoli gate is a generalization of the CNOT gate in the sense that it has two control qubits and one target qubit [1,13]. The target qubit flips if and only if the two control qubits are set. If we take qubit zero and qubit one as the control qubits and qubit two as the target qubit then we have

$$\begin{aligned} T_{210}|\Phi\rangle &= T_{210}(a_0|000\rangle + a_1|001\rangle + a_2|010\rangle + a_3|011\rangle + a_4|100\rangle + a_5|101\rangle + a_6|110\rangle + a_7|111\rangle) \\ &= a_0|000\rangle + a_1|001\rangle + a_2|010\rangle + a_7|011\rangle + a_4|100\rangle + a_5|101\rangle + a_6|110\rangle + a_3|111\rangle \\ &= a_0|0\rangle + a_1|1\rangle + a_2|2\rangle + a_7|3\rangle + a_4|4\rangle + a_5|5\rangle + a_6|6\rangle + a_3|7\rangle. \end{aligned} \quad (11)$$

Symbolically the Toffoli gate is represented by a vertical line connecting two dots (control qubits) and one cross (target qubit), as shown in Fig. 1(h).

## 3. Parallelization

### 3.1. General computational aspects

Computer memory and CPU time put limitations on the size of the quantum computer that can be simulated on a conventional digital computer. The required CPU time is mainly determined by the number of operations to be performed on the qubits. The CPU time does not put a hard limit on the simulation. However, the memory of the computer does. According to Eq. (2), the state of a  $L$ -qubit quantum computer is represented by a complex-valued vector of length  $D = 2^L$ . In view of the potentially large number of arithmetic operations, it is advisable to use 13–15 digit floating-point arithmetic (corresponding to 8 bytes for a real number). Thus, to represent a state of the quantum system of  $L$  qubits in a conventional, digital computer, we need at least  $2^{L+4}$  bytes. Hence, the amount of memory that is required to simulate a quantum computer with  $L$  qubits increases exponentially with the number of qubits  $L$ . For example, for  $L = 24$  ( $L = 36$ ) we need at least 256 MB (1 TB) of memory to store a single arbitrary state  $|\Phi\rangle$ .

As seen in Section 2, operations  $U$  on the state vector  $|\Phi\rangle$  result in a transformation of the amplitudes of the basis states in  $|\Phi\rangle$ . More specifically, let us denote

$$|\Phi\rangle = a(00\dots 0)|00\dots 0\rangle + a(0\dots 01)|0\dots 01\rangle + \dots + a(01\dots 1)|01\dots 1\rangle + a(11\dots 1)|11\dots 1\rangle, \quad (12)$$

and

$$|\Phi'\rangle = U|\Phi\rangle = a'(00\dots 0)|00\dots 0\rangle + a'(0\dots 01)|0\dots 01\rangle + \dots + a'(01\dots 1)|01\dots 1\rangle + a'(11\dots 1)|11\dots 1\rangle. \quad (13)$$

We first consider the single-qubit operations on qubit  $j$  that transform  $|\Phi\rangle$  in  $|\Phi'\rangle = U_j|\Phi\rangle$ . From Eq. (4), it follows that the Hadamard operation on qubit  $j$ ,  $H_j$ , transforms the amplitudes according to

$$\begin{aligned} a'(\dots * 0_j \dots) &= \frac{1}{\sqrt{2}}(a(\dots * 0_j \dots) + a(\dots * 1_j \dots)), \\ a'(\dots * 1_j \dots) &= \frac{1}{\sqrt{2}}(a(\dots * 0_j \dots) - a(\dots * 1_j \dots)), \end{aligned} \quad (14)$$

where we use the asterisk to indicate that the bits on the corresponding positions are the same. From Eq. (5), it follows that for  $X_j$  operating on  $|\Phi\rangle$  the elements of  $|\Phi'\rangle$  are obtained by

$$\begin{aligned} a'(\dots * 0_j \dots) &= \frac{1}{\sqrt{2}}(a(\dots * 0_j \dots) + ia(\dots * 1_j \dots)), \\ a'(\dots * 1_j \dots) &= \frac{1}{\sqrt{2}}(a(\dots * 1_j \dots) + ia(\dots * 0_j \dots)). \end{aligned} \quad (15)$$

In the case of  $|\Phi'\rangle = Y_j|\Phi\rangle$ , it follows from Eq. (6) that

$$\begin{aligned} a'(\dots * 0_j \dots) &= \frac{1}{\sqrt{2}}(a(\dots * 0_j \dots) + a(\dots * 1_j \dots)), \\ a'(\dots * 1_j \dots) &= \frac{1}{\sqrt{2}}(a(\dots * 1_j \dots) - a(\dots * 0_j \dots)). \end{aligned} \quad (16)$$

From Eq. (7) it follows that we obtain  $|\Phi'\rangle = R_j(\phi)|\Phi\rangle$  by leaving the amplitudes, for which the  $j$ th bit of their index is zero, unchanged and by multiplying the amplitudes, for which the  $j$ th bit of their index is one, with the phase factor  $e^{i\phi}$ . Hence we have,

$$\begin{aligned} a'(\dots * 0_j * \dots) &= (\dots * 0_j * \dots), \\ a'(\dots * 1_j * \dots) &= e^{i\phi} a(\dots * 1_j * \dots). \end{aligned} \quad (17)$$

In summary, performing an operation on qubit  $j$  requires in general an update of  $2^L$  elements of  $|\Phi\rangle$ . The single-qubit phase shift operation forms an exception and requires an update of  $2^{L-1}$  single amplitudes only. Note that all these operations can be done in place, that is, without using another vector of length  $2^L$ .

We now consider two-qubit operations  $|\Phi'\rangle = U_{kj}|\Phi\rangle$ , with  $j < k$ . For the CNOT operation  $\text{CNOT}_{kj}$ , where qubit  $j$  is the control qubit and qubit  $k$  is the target qubit, amplitudes for which bit  $j$  of their index is one and bit  $k$  of their index is zero need to be swapped with amplitudes for which bits  $j$  and  $k$  of their index are one (see Eq. (8)). We have

$$\begin{aligned} a'(\dots * 0_k * \dots * 0_j * \dots) &= a(\dots * 0_k * \dots * 0_j * \dots), \\ a'(\dots * 0_k * \dots * 1_j * \dots) &= a(\dots * 1_k * \dots * 1_j * \dots), \\ a'(\dots * 1_k * \dots * 0_j * \dots) &= a(\dots * 1_k * \dots * 0_j * \dots), \\ a'(\dots * 1_k * \dots * 1_j * \dots) &= a(\dots * 0_k * \dots * 1_j * \dots). \end{aligned} \quad (18)$$

For the controlled- $V$  operation  $V_{kj}$ , it follows from Eq. (10) that the amplitudes change according to the rules

$$\begin{aligned} a'(\dots * 0_k * \dots * 0_j * \dots) &= a(\dots * 0_k * \dots * 0_j * \dots), \\ a'(\dots * 0_k * \dots * 1_j * \dots) &= \frac{1}{2}[(1 + e^{i\phi})a(\dots * 0_k * \dots * 1_j * \dots) + (1 - e^{i\phi})a(\dots * 1_k * \dots * 1_j * \dots)], \\ a'(\dots * 1_k * \dots * 0_j * \dots) &= a(\dots * 1_k * \dots * 0_j * \dots), \\ a'(\dots * 1_k * \dots * 1_j * \dots) &= \frac{1}{2}[(1 - e^{i\phi})a(\dots * 0_k * \dots * 1_j * \dots) + (1 + e^{i\phi})a(\dots * 1_k * \dots * 1_j * \dots)]. \end{aligned} \quad (19)$$

Thus, performing two-qubit operations such as the CNOT and the controlled- $V$  amount to update  $2^{L-1}$  amplitudes.

For the Toffoli gate  $T_{lkj}$  ( $j < k < l$ ), where qubits  $j$  and  $k$  are the control qubits and qubit  $l$  is the target qubit, only  $2^{L-2}$  elements of  $|\Phi\rangle$  need to be updated, as can be seen from Eq. (11). The update rules read

$$\begin{aligned} a'(\dots * 0_l * \dots * 1_k * \dots * 1_j * \dots) &= a(\dots * 1_l * \dots * 1_k * \dots * 1_j * \dots), \\ a'(\dots * 1_l * \dots * 1_k * \dots * 1_j * \dots) &= a(\dots * 0_l * \dots * 1_k * \dots * 1_j * \dots), \end{aligned} \quad (20)$$

and the other amplitudes remain unchanged.

All the qubit operations discussed here can be carried out in  $\mathcal{O}(2^L)$  floating-point operations (here and in the sequel, if we count operations on a conventional computer, we make no distinction between operations such as add, multiply, or get from and put to memory). The qubit operations described earlier suffice to implement any unitary transformation on the state vector. As a matter of fact, the simple rules described are all that we need to simulate a universal quantum computer on a conventional computer.

The fact that both the amount of memory and arithmetic operations increase exponentially with the number of qubits is the major bottleneck for simulating quantum computers (or quantum systems in general) on a conventional computer. However, in contrast to the CPU time, the total amount of memory that is available on a computer puts a hard limit on the simulations that can be performed. From the user's perspective, the memory on a computer can be shared or distributed. On a shared memory computer, the state  $|\Phi\rangle$  of the quantum computer can be completely stored in the memory and all processors can access the entire memory. On a distributed memory machine, the elements of  $|\Phi\rangle$  are physically distributed over different nodes and each processor has direct access to its own local memory only. In the latter case, some extra programming is required to perform the communication between the processors. We use the standard Message Passing Interface (MPI) to perform the data communication [8]. We assume that the reader has some basic knowledge of MPI programming [8].

### 3.2. Implementation

Each MPI process has a rank and a local memory. We assume that the local memory can store the  $2^M$  amplitudes of the basis states of  $M$  qubits. Hence, to simulate a  $L$ -qubit quantum computer we need  $N = 2^L/2^M$  MPI processes. From Eq. (2), it is clear that the amplitudes  $a(x_{L-1} \dots x_0)$  where  $x_i = 0, 1$  for  $i = 0, \dots, L-1$ , can be stored at the local memory address  $A = \sum_{i=0}^{M-1} 2^i x_i$  of the MPI process with rank  $R = \sum_{i=M}^{L-1} 2^{i-M} x_i$ . In binary notation the local memory address and the rank of the processor reads  $A = (x_{M-1} \dots x_0)$  and  $R = (x_{L-1} \dots x_M)$ , respectively. Recall that the qubits are numbered from 0 to  $L-1$ , that is qubit 0 corresponds to the least significant bit of the integer index, running from zero to  $2^{L-1}$ , of the amplitude. An example for  $L = 4$  and  $M = 2$  is shown in the first four columns of Table 1. The MPI processes are labeled with their ranks running from 00 to 11. The memory addresses are also running from 00 to 11 for this example.

Table 1  
Single-qubit operation: Distribution of the amplitudes  $a(x_{L-1} \dots x_0)$  over the  $N$  MPI processes with  $2^M$  memory locations after application of the permutation  $\sigma_p$ , for the case  $L = 4$  and  $M = 2$  ( $N = 2^L/2^M$ )

|    | $\sigma_1 = \begin{pmatrix} 3 & 2 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{pmatrix}$ |           |           |           | $\sigma_2 = \begin{pmatrix} 3 & 2 & 1 & 0 \\ 3 & 0 & 1 & 2 \end{pmatrix}$ |           |           |           | $\sigma_3 = \begin{pmatrix} 3 & 2 & 1 & 0 \\ 2 & 0 & 1 & 3 \end{pmatrix}$ |           |           |           |
|----|---------------------------------------------------------------------------|-----------|-----------|-----------|---------------------------------------------------------------------------|-----------|-----------|-----------|---------------------------------------------------------------------------|-----------|-----------|-----------|
|    | 00                                                                        | 01        | 10        | 11        | 00                                                                        | 01        | 10        | 11        | 00                                                                        | 01        | 10        | 11        |
| 00 | $a(0000)$                                                                 | $a(0100)$ | $a(1000)$ | $a(1100)$ | $a(0000)$                                                                 | $a(0001)$ | $a(1000)$ | $a(1001)$ | $a(0000)$                                                                 | $a(0001)$ | $a(0100)$ | $a(0101)$ |
| 01 | $a(0001)$                                                                 | $a(0101)$ | $a(1001)$ | $a(1101)$ | $a(0100)$                                                                 | $a(0101)$ | $a(1100)$ | $a(1101)$ | $a(1000)$                                                                 | $a(1001)$ | $a(1100)$ | $a(1101)$ |
| 10 | $a(0010)$                                                                 | $a(0110)$ | $a(1010)$ | $a(1110)$ | $a(0010)$                                                                 | $a(0011)$ | $a(1010)$ | $a(1011)$ | $a(0010)$                                                                 | $a(0011)$ | $a(0110)$ | $a(0111)$ |
| 11 | $a(0011)$                                                                 | $a(0111)$ | $a(1011)$ | $a(1111)$ | $a(0110)$                                                                 | $a(0111)$ | $a(1110)$ | $a(1111)$ | $a(1010)$                                                                 | $a(1011)$ | $a(1110)$ | $a(1111)$ |

$\sigma_1$  corresponds to the identity permutation,  $\sigma_2$  interchanges local qubit 0 and nonlocal qubit 2,  $\sigma_3$  interchanges local qubit 2 and nonlocal qubit 3, after having interchanged qubits 0 and 2. The rank  $R$  of the MPI process is shown (in binary notation) in the second row. The local memory addresses are shown in the first column.

As can be seen from Eqs. (14)–(17), performing an operation on qubit  $j$  requires in general an update of  $2^L$  elements of  $|\Phi\rangle$ . A qubit  $j$  for which  $j < M$  we call a “local” qubit. In the example shown in the first four columns of Table 1 qubits 0 and 1 are local qubits. Updating the amplitudes is easy if the qubit is local because this requires no communication between the MPI processes.

Similarly, if  $j \geq M$  we call qubit  $j$  a “nonlocal” qubit. An operation on qubit  $j$  requires communication between the MPI processes. In the example shown in the first four columns of Table 1, qubits 2 and 3 are nonlocal. Note that the nonlocal qubits form the binary representation of the rank of the corresponding MPI process and that the local qubits form the binary representation of the memory address. An obvious way of communication would be for pairs of MPI processes to first interchange one half of their data. Then, the operations on the amplitudes can be performed as if the qubit was local. Finally, the MPI processes need again to interchange the (modified) data. A clear drawback of this method is that half of the amplitudes needs to be interchanged twice for every single-qubit operation on qubit  $j$  for which  $j \geq M$ .

In order to reduce the amount of communication between the MPI processes we use a different method to handle the operation on nonlocal qubits. We introduce a permutation  $\sigma: \{L-1, \dots, 0\} \rightarrow \{\sigma(L-1), \dots, \sigma(0)\}$  of the  $L$  qubits. We denote the permutation by a matrix with two rows and  $L$  columns. The first row contains integer numbers ordered from  $L-1$  to 0. These numbers correspond to the position of the bits in the index of the amplitude. The second row also contains integer numbers ranging from  $L-1$  to 0 but they are not necessarily ordered. These numbers refer to the qubits. Local qubit  $k$  corresponds to bit  $l = \sigma^{-1}(k) < M$  of the index of the amplitude and nonlocal qubit  $m$  corresponds to bit  $n = \sigma^{-1}(m) \geq M$  of the index of the amplitude. After applying the permutation  $\sigma$  the amplitudes  $a(x_{L-1} \dots x_0)$  ( $x_i = 0, 1$ ) are stored at the address  $A = \sum_{i=0}^{M-1} 2^i x_{\sigma(i)}$  of the local memory assigned to the MPI process with rank  $R = \sum_{i=M}^{L-1} 2^{i-M} x_{\sigma(i)}$ . In binary notation we have  $A = (x_{\sigma(M-1)} \dots x_{\sigma(0)})$  and  $R = (x_{\sigma(L-1)} \dots x_{\sigma(M)})$ . Note that after applying the permutation  $\sigma$ , the nonlocal qubits still form the binary representation of the rank of the corresponding MPI process. Similarly, the local qubits still form the binary representation of the address.

Logically, to make a nonlocal qubit  $m$  local, all we have to do is to select a permutation that interchanges a local qubit, say  $k$ , and the nonlocal qubit  $m$  and leaves the other qubits in place. Clearly, it is always easy to find such a permutation. We now consider what it actually means for the parallel machine to perform an interchange of a local and nonlocal qubit. A simple reshuffling of the corresponding amplitudes would work but is not very efficient: We would like to minimize the amount of interprocess communication.

In terms of data transfer between MPI processes, a permutation that interchanges local qubit  $k$  and nonlocal qubit  $m$ , swaps the amplitudes with addresses

$$\begin{aligned}
 A &= (* \dots * 0_l * \dots *) \quad \text{of MPI process with rank } R = (* \dots * 1_n * \dots *), \\
 A &= (* \dots * 1_l * \dots *) \quad \text{of MPI process with rank } R = (* \dots * 0_n * \dots *),
 \end{aligned} \tag{21}$$

where we use the asterisk to indicate that the bits on the corresponding position are the same. Hence, in this operation, only half of the amplitudes in the memory of an MPI process are swapped against half of the amplitudes in the memory of another MPI process. After this swap operation, the previously nonlocal qubit  $m$  has become local, and we can carry out the operations on this qubit in exactly the same way as we did for the originally local qubits, that is fully parallel. Using this scheme we do not need to send the modified amplitudes back to the MPI processes from which they originate as the permutation keeps track of the memory addresses of the amplitudes.

An example of this process of swapping data is shown in Table 1, for  $L = 4$ ,  $M = 2$ . The permutations  $\sigma$  that have been applied are shown on the top row. The four columns below  $\sigma_1$  show the original content of the local memories. In this example, qubits 0 and 1 are local and qubits 2 and 3 are nonlocal. Hence, operations on qubit 0 or 1 can be performed in parallel by each MPI process. However, operations on qubit 2 and 3 require communication between the MPI processes. Let us now assume that we want to carry out an operation on qubit 2. According to our scheme, this requires that the pairs of amplitudes  $a(x_3, 0_2, x_1, x_0)$  and



Table 2

Two-qubit operation: Distribution of the amplitudes  $a(x_{L-1} \dots x_0)$  over the  $N$  MPI processes with  $2^M$  memory locations after application of the permutation  $\sigma_p$ , for the case  $L = 4$  and  $M = 2$  ( $N = 2^L/2^M$ )

|    | $\sigma_1 = \begin{pmatrix} 3 & 2 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{pmatrix}$ |           |           |           | $\sigma_2 = \begin{pmatrix} 3 & 2 & 1 & 0 \\ 1 & 0 & 3 & 2 \end{pmatrix}$ |           |           |           |
|----|---------------------------------------------------------------------------|-----------|-----------|-----------|---------------------------------------------------------------------------|-----------|-----------|-----------|
|    | 00                                                                        | 01        | 10        | 11        | 00                                                                        | 01        | 10        | 11        |
| 00 | $a(0000)$                                                                 | $a(0100)$ | $a(1000)$ | $a(1100)$ | $a(0000)$                                                                 | $a(0001)$ | $a(0010)$ | $a(0011)$ |
| 01 | $a(0001)$                                                                 | $a(0101)$ | $a(1001)$ | $a(1101)$ | $a(0100)$                                                                 | $a(0101)$ | $a(0110)$ | $a(0111)$ |
| 10 | $a(0010)$                                                                 | $a(0110)$ | $a(1010)$ | $a(1110)$ | $a(1000)$                                                                 | $a(1001)$ | $a(1010)$ | $a(1011)$ |
| 11 | $a(0011)$                                                                 | $a(0111)$ | $a(1011)$ | $a(1111)$ | $a(1100)$                                                                 | $a(1101)$ | $a(1110)$ | $a(1111)$ |

$\sigma_1$  corresponds to the identity permutation,  $\sigma_2$  interchanges local qubit 0 with nonlocal qubit 2, and local qubit 1 with nonlocal qubit 3. The rank  $R$  of the MPI process is shown (in binary notation) in the second row. The local memory addresses are shown in the first column.

$a(x_3, x_2, x_1, x_0)$  reside in the same local memory. Conceptually, this can be accomplished by rearranging the amplitudes over the local memories according to the permutation  $\sigma_2$  (see Table 1) whereas the actual data exchange is carried out according to Eq. (21).

Now qubits 2 and 1 are local and qubits 0 and 3 are nonlocal. Assume that we now want to operate on qubit 3 which is currently nonlocal. Then, we may interchange, for example, local qubit 2 with the nonlocal qubit 3. This can be accomplished by applying the permutation  $\sigma_3$  (see Table 1) and data exchange rule Eq. (21). At this point, qubits 3 and 1 are local and qubits 0 and 2 are nonlocal.

To summarize: A single-qubit operation on a nonlocal qubit consists of a local-nonlocal data exchange defined by Eq. (21), followed by the actual unitary operation. By construction, there is no interprocess communication during and after the latter. If the qubit is local, we simply skip the step of exchanging data. From Eq. (17), it is clear that the single-qubit phase-shift operation never requires communication between MPI processes. Thus, for this particular operation there is additional room for optimization. In our present simulation code, we have chosen not to optimize gate operations on this level. Optimization of parallel code for specific gates will be dealt with in depth in a subsequent publication [4].

Extending the single-qubit scheme to two and three-qubit operations is conceptually straightforward but the rules to determine which processes have to exchange data become more complicated. From Eqs. (18) and (19), it follows that a two-qubit operation involves an update of  $2^{L-1}$  elements of  $|\Phi\rangle$ . If these elements are located in the same local memory, that is if both qubits are local, no communication between the MPI processes is required and the operation can be performed independently by all the MPI processes. However, if one of the qubits is nonlocal, some communication is necessary. If only one qubit is nonlocal, we can use the same procedure as described above to interchange the qubit for a local one. If both qubits are nonlocal, we can also use the same procedure, but twice. For each qubit interchange, half of the amplitudes in the memory of a MPI process is swapped against half of the amplitudes in the memory of another MPI process. Hence, this scheme results in a full memory swap. However, this amount of communication can be reduced by exchanging two nonlocal qubits and two local qubits simultaneously. We illustrate this by the example for the case  $L = 4$ ,  $M = 2$ , shown in Table 2. The four columns below  $\sigma_1$  show the original content of the local memory. In this example, qubits 0 and 1 are local and qubits 2 and 3 are nonlocal. Hence, two-qubit operations on qubit 0 and 1 can be performed independently by each MPI process. However, all other two-qubit operations require communication between the MPI processes.

Let us now assume that we want to do a two-qubit operation on qubits 2 and 3. Permutation  $\sigma_2$  interchanges nonlocal qubits 2 and 3 with local qubits 0 and 1, respectively. From Table 2 we see that each MPI process has to communicate with three other processes. For example, MPI process 00 keeps the amplitude in address 00, interchanges its amplitude in address 01 with the amplitude in address 00 of MPI process 01, interchanges its amplitude in address 10 with the amplitude in address 00 of MPI process 10, and interchanges its amplitude in address 11 with the amplitude in address 00 of MPI process 11.

In general, for a two-qubit interchange, the MPI processes communicate in groups of four (in the example of Table 2 there is only one group of four). Each MPI process keeps one quarter of the amplitudes in its local memory and interchanges the other three quarters of amplitudes with amplitudes in the local memories of the other MPI processes in the group. Swapping of the amplitudes can be accomplished by applying the permutation  $\sigma : \{L-1, \dots, 0\} \rightarrow \{\sigma(L-1), \dots, \sigma(0)\}$  of the  $L$  qubits so that the MPI processes interchange local qubits  $k_1, k_2$  for nonlocal qubits  $m_1, m_2$ . Local qubit  $k_i$  corresponds to bit  $l_i = \sigma^{-1}(k_i) < M$  of the index of the amplitude and nonlocal qubit  $m_i$  corresponds to bit  $n_i = \sigma^{-1}(m_i) \geq M$  of the index of the amplitude, where  $i = 1, 2$ . Adopting the convention that  $l_2 > l_1$  and  $m_2 > m_1$ , interchanging local qubits  $k_1, k_2$  and nonlocal qubits  $m_1, m_2$  amounts to swapping the amplitudes with local memory addresses

$$\begin{aligned} A &= (* \dots * 0_{l_2} * \dots * 0_{l_1} * \dots *) \quad \text{of MPI process with rank } R = (* \dots * 0_{n_2} * \dots * 1_{n_1} * \dots *), \\ A &= (* \dots * 0_{l_2} * \dots * 1_{l_1} * \dots *) \quad \text{of MPI process with rank } R = (* \dots * 0_{n_2} * \dots * 0_{n_1} * \dots *), \end{aligned} \quad (22)$$

$$\begin{aligned} A &= (* \dots * 0_{l_2} * \dots * 0_{l_1} * \dots *) \quad \text{of MPI process with rank } R = (* \dots * 1_{n_2} * \dots * 0_{n_1} * \dots *), \\ A &= (* \dots * 1_{l_2} * \dots * 0_{l_1} * \dots *) \quad \text{of MPI process with rank } R = (* \dots * 0_{n_2} * \dots * 0_{n_1} * \dots *), \end{aligned} \quad (23)$$

$$\begin{aligned}
 A &= (* \cdots * 0_{l_2} * \cdots * 0_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 1_{n_2} * \cdots * 1_{n_1} * \cdots *), \\
 A &= (* \cdots * 1_{l_2} * \cdots * 1_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 0_{n_2} * \cdots * 0_{n_1} * \cdots *),
 \end{aligned} \tag{24}$$

$$\begin{aligned}
 A &= (* \cdots * 0_{l_2} * \cdots * 1_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 1_{n_2} * \cdots * 0_{n_1} * \cdots *), \\
 A &= (* \cdots * 1_{l_2} * \cdots * 0_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 0_{n_2} * \cdots * 1_{n_1} * \cdots *),
 \end{aligned} \tag{25}$$

$$\begin{aligned}
 A &= (* \cdots * 0_{l_2} * \cdots * 1_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 1_{n_2} * \cdots * 1_{n_1} * \cdots *), \\
 A &= (* \cdots * 1_{l_2} * \cdots * 1_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 0_{n_2} * \cdots * 1_{n_1} * \cdots *),
 \end{aligned} \tag{26}$$

$$\begin{aligned}
 A &= (* \cdots * 1_{l_2} * \cdots * 0_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 1_{n_2} * \cdots * 1_{n_1} * \cdots *), \\
 A &= (* \cdots * 1_{l_2} * \cdots * 1_{l_1} * \cdots *) \quad \text{of MPI process with rank } R = (* \cdots * 1_{n_2} * \cdots * 0_{n_1} * \cdots *).
 \end{aligned} \tag{27}$$

As before, the asterisks in Eqs. (22)–(27) indicate that the bits on the corresponding position are the same. As in the case of the single-qubit operations, particular two-qubit operations (such as the controlled phase shift) allow for additional optimization, but we have chosen not to do so because we wanted to implement all two-qubit operations in the same manner.

The algorithm to interchange a pair of local qubits with a pair of nonlocal qubits Eqs. (22)–(27) can be generalized to swap as many local qubits with nonlocal qubits, with the restriction that the number of qubits to be swapped cannot exceed the number of local or nonlocal qubits. Exchanging  $K$  pairs of local and nonlocal qubits simultaneously requires each MPI process to send  $(2^K - 1)2^M/2^K$  amplitudes to another MPI process. Sequentially exchanging the  $K$  pairs amounts to sending  $K2^M/2$  amplitudes to another process. Thus, exchanging  $K$  pairs of local and nonlocal qubits simultaneously involves less communication. However, the more qubits we swap simultaneously, the larger becomes the group of MPI processes that communicate with each other. If this number becomes too big the computer might hang in network collisions. So, in practice there is a limit to the choice of  $K$ . In our implementation of the algorithm,  $K$  is an input variable. For each MPI process, we first compute the ranks of the other MPI processes with which the MPI process has to communicate. We do this on the basis of the  $K$  pairs of local and nonlocal qubits that need to be swapped. Before exchanging data between the MPI processes belonging to one group, we fill a buffer for each MPI process of the group. The buffer contains all amplitudes that need to be send from one MPI process to the other MPI processes of the group. In order to reduce the memory allocation for the buffers, an essential requirement for simulating quantum computers with a large number of qubits  $L$ , we split the MPI send instruction in a fixed (user controlled), smaller number of send instructions.

#### 4. Simulator

We have implemented the algorithm described in Section 3 in Fortran 90. The computer code contains all quantum operations that are necessary for universal quantum computation and runs on machines with distributed and/or shared memory. Standard MPI is used for interprocess communication. Optionally, OpenMP can be used for parallel processing within each MPI process. This computer code forms the engine of the parallel quantum computer simulator. It compiles (without modifications) and runs on various computer architectures, ranging from PCs to high-end (vector) parallel machines. The number of qubits in the computer codes is limited to 62, but in practice the maximum number of qubits is set by the memory of the machine on which the code runs.

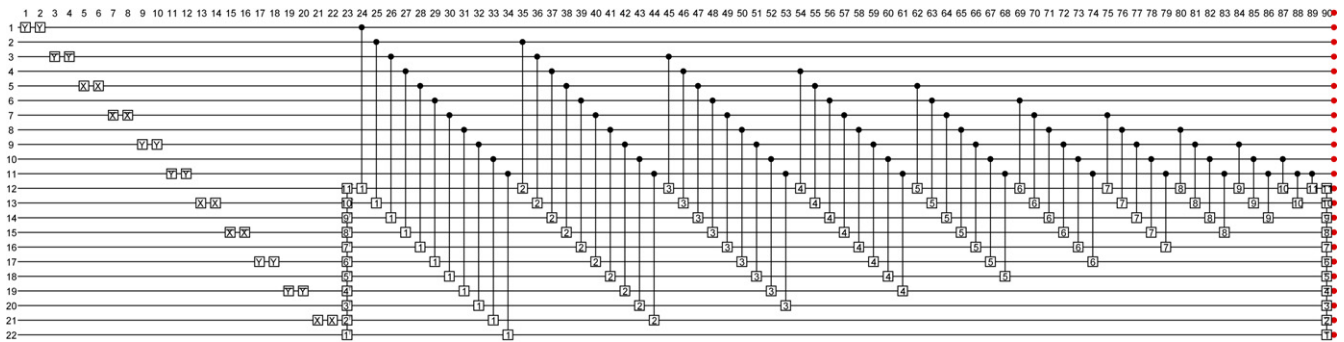


Fig. 2. Quantum network to add the content of two 11-qubit registers modulo  $2^{11}$ . The horizontal lines, numbered from 1 to 22, denote the qubits involved in the quantum operations. The horizontal row of numbers labels the quantum operations. The first 22 quantum operations are the single-qubit operations  $XX$ ,  $YY$ ,  $\bar{X}\bar{X}$  and  $\bar{Y}\bar{Y}$  that are used to put numbers (in binary notation) in the registers. The first register (qubits 1 to 11) contains the number 1365 and the second one (qubits 12 to 22) contains the number 682. Quantum operation 23 is a quantum Fourier transform on the second register. The full quantum network for this quantum Fourier transform is depicted in Fig. 3. Quantum operations 24–89 are controlled phase shift operations that add the information contained in the first register to the content of the second one. Quantum operation 90 is an inverse quantum Fourier transform. The (red) dots at the end of each horizontal line denote the measurement process at each qubit. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)



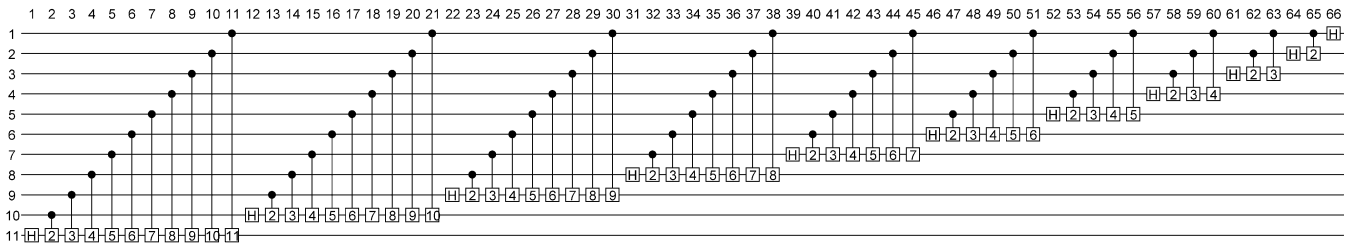


Fig. 3. Quantum network, built from Hadamard gates and single-qubit phase shifts, to perform a quantum Fourier transform on eleven qubits [1].

The simulator takes as input the description of a quantum network in terms of pseudocode (a text-formatted file). Quantum networks are drawn by making use of a graphical user interface. For this purpose, we developed Quantum Computer Circuit Editor (QCCE), a Microsoft Windows application. QCCE contains graphical symbols for each of the qubit operations described in Section 2 and for the quantum Fourier transform. Examples of quantum networks drawn by QCCE are shown in Figs. 2 and 3. QCCE also contains an interpreter that takes the quantum network as input and generates a text-formatted file that contains pseudocode describing all operations to be performed on the qubits, including the swap operations described in Section 3.2. An example of the pseudocode is given in Appendix A. Since drawing a quantum network for a large quantum computer can be quite cumbersome, it is often more efficient to write a dedicated program that generates the pseudocode language directly. Since the quantum computer simulator software is a standalone application that takes ASCII files as input, any software package (such as <http://www.phys.cs.is.nagoya-u.ac.jp/~watanabe/qcad/index.html>) that can draw quantum networks may be used, as long as it generates the input files (including the swap operations) in the format that the simulator can understand.

## 5. Simulation results

In this section we present and discuss the results of running the massive parallel quantum computer simulator on various parallel computers.

The computers on which we perform the simulations and their characteristics are listed in Table 3. On all computers we have tested pure OpenMP, pure MPI and combined MPI/OpenMP code. In practice it turns out that the OpenMP code is up to a factor of two slower than the pure MPI code. As a consequence, also the MPI/OpenMP code is slower than the pure MPI code. A detailed study of the (dis)advantages of this hybrid approach in the case of optimized parallel code for specific gates will be presented in a future publication [4]. In what follows we only present the results of the pure MPI code. Because of the special architecture of the Cray X1E processors (Cray Multistreaming Processors (MSPs) with various vector pipes per MSP), some CRAY specific directives have been added to the code.

A rather simple algorithm to test the scaling properties of the simulation software with the number of qubits is to perform a Hadamard operation on each qubit of a  $L$ -qubit quantum computer. However, simulating gates for single-, two- and three-qubit operations only is not sufficient to test the correctness of quantum computer simulation software and to compare the performance of various massive parallel computers. Hence, for this purpose, we consider some more sophisticated quantum algorithms. A first nontrivial example is the qubit adder that adds the content of several qubit registers. This quantum algorithm is built up from several quantum operations, including a quantum Fourier transform, and has the advantage that it is very easy to check the correctness of the simulation result.

Fig. 2 shows the quantum network to add the content of two eleven-qubit registers. Note that here the qubits are numbered starting from one. We use a similar modular structure as was used for adding the content of three four-qubit registers [14]. The basic idea of the algorithm is to use the Quantum Fourier Transform (QFT) to first transfer the information in one register to the phase factors of the amplitudes, then use controlled phase shifts to add the information from the other registers, and finally QFT

Table 3  
Overview of the computer systems used for testing the parallel quantum computer software

|           | SGI Altix 3700  | IBM Regatta p690+ | IBM Blue Gene/L | Hitachi SR11000/J1 | Cray X1E            |
|-----------|-----------------|-------------------|-----------------|--------------------|---------------------|
| Location  | SARA            | FZ-J              | RuG             | ISSP               | CRAY Inc.           |
| Memory    | 832 GB          | 5.2 TB            | 3.1 TB          | 2.8 TB             | 512 GB              |
| # CPUs    | 416             | 1312              | 12288           | 2048               | 128                 |
| CPU type  | Intel Itanium 2 | Power 4+          | Power PC 970    | POWER5             | 64-bit Cray X1E MSP |
| CPU clock | 1.3 GHz         | 1.7 GHz           | 0.7 GHz         | 1.9 GHz            | 1.13 GHz            |

Simulations have been performed on machines located at SARA Computing and Networking Services, Amsterdam, The Netherlands (SARA); Forschungszentrum Jülich, Jülich, Germany (FZ-J); Computer Center, University of Groningen, Groningen, The Netherlands (RuG); The Institute for Solid State Physics (ISSP) of the University of Tokyo, Tokyo, Japan (ISSP); Cray Inc., Seattle, USA (Cray Inc.).

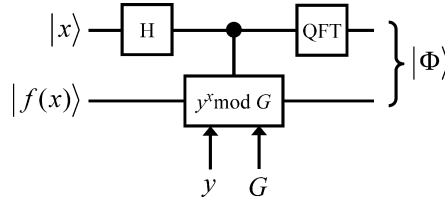


Fig. 4. Schematic diagram of Shor's algorithm.

back to the original representation. These quantum networks perform addition modulo two to the power of the number of qubits in the register on which the QFT is being applied.

Originally all qubits are in the state  $|0\rangle$ . The single-qubit operations  $XX$ ,  $YY$ ,  $\bar{X}\bar{X}$  and  $\bar{Y}\bar{Y}$  are used to put numbers (in binary notation) in the registers. Note that all these operations bring a qubit in the state  $|1\rangle$ . Hence, in principle we could use only one of these operations to put numbers in the registers. However, in order to test the various single-qubit operations we used the four different operations. In the example shown in Fig. 2 the first register (qubits 1 to 11) contains the number 1365 and the second register (qubits 12 to 22) the number 682. We perform a QFT on the second register to transfer the information in the second register to the phase factors of the amplitudes. The full quantum network for a QFT on eleven qubits is shown in Fig. 3. After applying the QFT to the second register, we use controlled phase shifts to add the information from the first register to the content of the second one. Finally we QFT back to the original representation.

As another nontrivial test we implement Shor's algorithm on a quantum computer containing up to 36 qubits. Briefly, Shor's algorithm finds the prime factors  $p$  and  $q$  of a composite integer  $G = p \times q$  by determining the period of the function  $f(x) = y^x \bmod G$  for  $x = 0, 1, \dots$ . Here,  $1 < y < G$  should be coprime (greatest common divisor of  $y$  and  $G$  is 1) to  $G$  (otherwise, since  $G = p \times q$ ,  $y = p$  or  $y = q$ , so we already guessed the solution). Let  $r$  denote the period of  $f(x)$ , that is  $f(x) = f(x + r)$ . If the chosen value of  $y$  yields an odd period  $r$ , we repeat the algorithm with another choice for  $y$ , until we find an  $r$  that is even. Once we have found an even period  $r$ , we compute  $y^{r/2} \bmod G$ . If  $y^{r/2} \not\equiv \pm 1 \bmod G$ , then we find the factors of  $G$  by calculating the greatest common divisors of  $y^{r/2} \pm 1$  and  $G$ .

On an ideal quantum computer, this algorithm can be carried out with a number of unitary operations that increases as a polynomial, not as the exponential, of the number of qubits required to store the number  $G$ . The schematic diagram of Shor's algorithm is shown in Fig. 4. The quantum computer has  $L$  qubits. There are two qubit registers: A  $x$ -register with  $X$  qubits to hold the values of  $x$  and a  $f$ -register with  $F = L - X$  qubits to hold the values of  $f(x) = y^x \bmod G$ . What is the largest number we can factorize on a quantum computer with  $L$  qubits? If we write the number  $G$  in its binary representation  $G = \sum_{i=0}^{g_1-1} 2^i n_i$ , where  $n_i = 0, 1$ , then it is clearly seen that  $F = g_1$ . For Shor's algorithm to work properly, that is to find the correct period  $r$  of  $f(x)$ , the number of qubits  $X$  in the  $x$ -register should satisfy [15]

$$G^2 \leq 2^X \leq 2G^2. \quad (28)$$

Writing  $G^2 = \sum_{i=0}^{g_2-1} 2^i m_i$ , where  $m_i = 0, 1$ , it follows from Eq. (28) that  $g_2 - 1 \leq X \leq g_2 + 1$ . Omitting numbers  $G$  that can be written as a power of two (which are trivial to factorize), the smallest value for  $X$  (and hence the largest value for  $F$ ) is given by  $g_2$ . Hence,  $L = g_1 + g_2$ . Since either  $g_2 = 2g_1 - 1$  or  $g_2 = 2g_1$ , it follows that the maximum number of qubits that can be reserved for the  $f$ -register is given by  $F = g_1 = \lfloor (L + 1)/3 \rfloor$ . For example, on a 36-qubit quantum computer  $G = 4087 = 61 \times 67 \leq 2^{12}$  is the largest integer that can be factorized by Shor's algorithm.

The initial state of the machine is  $|\Phi_0\rangle = |0\rangle$ . After applying Hadamard operations to all qubits of the  $x$ -register we have

$$|\Phi_1\rangle = 2^{-X/2} \sum_{x=0}^{2^X-1} |x\rangle_x |0\rangle_f, \quad (29)$$

where  $|\cdot\rangle_x$  and  $|\cdot\rangle_f$  denote the state of the  $x$  and  $f$ -register, respectively. Then, we compute  $f(x)$  for each value  $x = 0, \dots, X - 1$ . This is implemented as a conditional operation, as indicated by the middle box in Fig. 4. After the second step, the quantum computer is in the state

$$|\Phi_2\rangle = 2^{-X/2} \sum_{x=0}^{2^X-1} |x\rangle_x |f(x)\rangle_f. \quad (30)$$

The period  $r$  of  $f(x)$  can now be determined by applying a Fourier transformation, not to  $|f(x)\rangle_f$  but to  $|x\rangle_x$ , as indicated in Fig. 4. To see how this step works, we use the periodicity of  $f(x)$  to rewrite Eq. (30) as

$$2^{-X/2} \sum_{x=0}^{2^X-1} |x\rangle_x |f(x)\rangle_f = 2^{-X/2} \sum_{x=0}^{r-1} (|x\rangle_x + |x+r\rangle_x + \dots) |f(x)\rangle_f. \quad (31)$$

Using the Fourier representation of  $|x\rangle_x$  we obtain

$$\begin{aligned} |\Phi_3\rangle &= 2^{-X/2} \sum_{x=0}^{2^X-1} |x\rangle_x |f(x)\rangle_f \\ &= 2^{-X} \sum_{k=0}^{2^X-1} \sum_{x=0}^{r-1} e^{2\pi i k x / 2^X} (1 + e^{2\pi i k r / 2^X} + e^{4\pi i k r / 2^X} + \dots + e^{2\pi i k r (s-1) / 2^X}) |k\rangle_x |f(x)\rangle_f \\ &\quad + 2^{-X} \sum_{k=0}^{2^X-1} \sum_{x=0}^{s-1} e^{2\pi i k x / 2^X} e^{2\pi i k r s / 2^X} |k\rangle_x |f(x)\rangle_f, \end{aligned} \quad (32)$$

where  $s = \lfloor 2^X / r \rfloor$  denotes the largest integer  $s$  such that  $rs \leq 2^X$ . The probability to observe the quantum computer in the state  $|k\rangle$  reads

$$p_k(r) = \frac{r}{2^{2X}} \left( \frac{\sin(\pi k r s / 2^X)}{\sin(\pi k r / 2^X)} \right)^2 - \frac{2^X - rs}{2^{2X}} \frac{\sin(\pi k r (2s+1) / 2^X)}{\sin(\pi k r / 2^X)}. \quad (33)$$

The function  $p_k(r)$  is strongly peaked for all  $kr \approx 2^X$ . The probability to observe the machine in the state  $|k\rangle$  for which  $kr \approx 2^X$  is  $p_k(r) \approx r^{-1}$  [16]. Given the observed state  $|k\rangle$ , we can find  $r$  because the condition Eq. (28) guarantees that there exists exactly one function  $k'/r$  that satisfies [15]

$$\left| \frac{k}{2^X} - \frac{k'}{r} \right| \leq \frac{1}{2^{X+1}}. \quad (34)$$

The fraction  $k'/r$  (with  $r < G$ ) can be found effectively by computing the convergents of the continued fraction representation of  $k/2^X$  [1,15–17]. From Eq. (33) we can easily compute the expectation values of the qubits of the  $x$ -register. Let  $Q_i |x_i\rangle = x_i |x_i\rangle$  for

Table 4  
Representative results of running Shor's algorithm on the massive parallel quantum computer simulator

|                          |                      |                       |                       |                       |                        |
|--------------------------|----------------------|-----------------------|-----------------------|-----------------------|------------------------|
| $L$                      | 24                   | 33                    | 33                    | 36                    | 36                     |
| $X$                      | 16                   | 22                    | 22                    | 24                    | 24                     |
| $G$                      | $247 = 13 \times 19$ | $1961 = 37 \times 53$ | $2047 = 23 \times 89$ | $4087 = 61 \times 67$ | $4033 = 37 \times 109$ |
| $y$                      | 194                  | 1698                  | 617                   | 1392                  | 1693                   |
| $r$                      | 18                   | 468                   | 88                    | 44                    | 108                    |
| $\langle Q_0 \rangle$    | 0.500                | 0.500                 | 0.500                 | 0.500                 | 0.500                  |
| $\langle Q_1 \rangle$    | 0.500                | 0.500                 | 0.500                 | 0.500                 | 0.500                  |
| $\langle Q_2 \rangle$    | 0.500                | 0.500                 | 0.500                 | 0.500                 | 0.500                  |
| $\langle Q_3 \rangle$    | 0.445                | 0.500                 | 0.461                 | 0.461                 | 0.500                  |
| $\langle Q_4 \rangle$    | 0.445                | 0.500                 | 0.459                 | 0.459                 | 0.490                  |
| $\langle Q_5 \rangle$    | 0.445                | 0.500                 | 0.455                 | 0.455                 | 0.482                  |
| $\langle Q_6 \rangle$    | 0.444                | 0.499                 | 0.455                 | 0.455                 | 0.482                  |
| $\langle Q_7 \rangle$    | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.482                  |
| $\langle Q_8 \rangle$    | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.482                  |
| $\langle Q_9 \rangle$    | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{10} \rangle$ | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{11} \rangle$ | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{12} \rangle$ | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{13} \rangle$ | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{14} \rangle$ | 0.444                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{15} \rangle$ | 0.500                | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{16} \rangle$ |                      | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{17} \rangle$ |                      | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{18} \rangle$ |                      | 0.496                 | 0.455                 | 0.455                 | 0.481                  |
| $\langle Q_{19} \rangle$ |                      | 0.496                 | 0.500                 | 0.455                 | 0.481                  |
| $\langle Q_{20} \rangle$ |                      | 0.500                 | 0.500                 | 0.455                 | 0.481                  |
| $\langle Q_{21} \rangle$ |                      | 0.500                 | 0.500                 | 0.455                 | 0.481                  |
| $\langle Q_{22} \rangle$ |                      |                       |                       | 0.500                 | 0.500                  |
| $\langle Q_{23} \rangle$ |                      |                       |                       | 0.500                 | 0.500                  |

The total number of qubits of the simulator is given by  $L$  and  $X$  is the number of qubits in the  $x$ -register. The remaining  $L - X$  qubits are used to hold the results of  $y^x \bmod G$ , where  $G$  is the composite integer to be factorized. The values of  $1 < y < G$  are chosen such that the period  $r$  is even and  $y^{r/2} \neq \pm 1 \bmod G$ . The rows labeled by  $\langle Q_i \rangle$  for  $i = 0, \dots, X - 1$  give the simulation results for the expectation values of the  $X$  qubits. These results are the same as those obtained from the analytical expression Eq. (35).  $L = 24$ : Thinkpad T43P (number of MPI processes  $N = 1$ , elapsed time  $t_E = 41$  s);  $L = 33$ : SGI Altix 3700 ( $N = 64$ ,  $t_E = 630$  s), IBM Regatta p690+ ( $N = 64$ ,  $t_E = 810$  s);  $L = 36$ : IBM Regatta p690+ ( $N = 512$ ,  $t_E = 970$  s) and IBM Blue Gene/L ( $N = 4096$ ,  $t_E = 170$  s).

$x_i = 0, 1$ , then

$$\begin{aligned} \langle Q_i \rangle &= \langle \Phi_3 | Q_i | \Phi_3 \rangle = \sum_{k,k'} \langle \Phi_3 | k' \rangle \langle k' | Q_i | k \rangle \langle k | \Phi_3 \rangle \\ &= \sum_{k,k'} \langle \Phi_3 | k' \rangle \delta_{k,k'} \langle k | \Phi_3 \rangle \langle k | Q_i | k \rangle = \sum_k p_k(r) k_i, \end{aligned} \quad (35)$$

where  $k_i$  denotes the  $i$ th bit of  $k$ . The quantum computer simulator should reproduce the values of all  $\langle Q_i \rangle$ , for  $i = 0, \dots, X - 1$ , otherwise there is definitely something wrong in the simulation. On the other hand, agreement is not a guarantee that the simulator is free of errors. Obviously, for a  $L = 36$  qubit simulation, storing the final state for further analysis requires a lot ( $> 1$  TB) of disk space. To alleviate this problem, we have added to the simulator a procedure that takes the final state as input and generates a user-specified number of basis states  $|x\rangle$  with probability  $|\langle x | \Phi_3(x) \rangle|^2$ . This procedure is also fully parallel and uses all available MPI processes.

In Table 4 we show the results of running Shor's algorithm on the massive parallel computer simulator. The example for  $L = 24$  qubits was run on an IBM Thinkpad T43P. The two examples for  $L = 33$  qubits were both run on the SGI Altix 3700 and on the IBM Regatta p690+. The two examples for  $L = 36$  are both run on the IBM Regatta p690+ and on the IBM Blue Gene/L. All the tests described in this section confirm that the simulator is producing correct results.

## 6. Benchmark results

The quantum algorithms that we use for benchmarking the simulator software are the Hadamard operation performed on each qubit, qubit adds to add the content of three 11-qubit registers, two 17-qubit registers, five 7-qubit registers and three 12-qubit registers. Tables 5–9 of Appendix B summarize the results on the various machines. In order to compare the performance of the different machines we use the data from Tables 5–9 and plot them in Figs. 5–7. Fig. 5 depicts the elapsed time  $t_E$  divided by the CPU time  $t_{CPU}$  and multiplied by the number of MPI processes  $N$  as a function of the number of qubits  $L$ . All data listed in Tables 5–9 is processed. In the ideal case, we expect that  $N \times t_E / t_{CPU} = 1$ , independent of  $L$ . As can be seen from Fig. 5, on most machines we observe scaling properties that are very close to ideal. Deviations from the ideal behavior, if there are any, are relatively small. The IBM Regatta p690+ and the Cray X1E show the largest deviations.

In Fig. 6 we show the CPU time for a Hadamard operation  $H_n$  carried out on qubit  $n$  of a quantum computer with  $L = 34$  qubits. In this case the CPU time is measured per gate and does not include the time for measuring the qubit. Recall that we number the qubits starting from zero, so that  $0 < n \leq 33$ . We only show results for the IBM Regatta p690+ (diamonds), IBM Blue Gene/L (flipped triangles) and the Hitachi SR11000/J1 (circles). The number of MPI processes on the IBM Regatta p690+, the IBM Blue Gene/L and the Hitachi SR11000/J1 are  $N = 128$ ,  $N = 1024$  and  $N = 128$ , respectively. Hence, on the IBM Regatta p690+ and the Hitachi SR11000/J1 qubits  $0 \leq n \leq 26$  are local and on the IBM Blue Gene/L qubits  $0 \leq n \leq 23$  are local. In all cases, the CPU time for  $H_n$  carried out on a nonlocal qubit is equal to the CPU time for the Hadamard gate performed on qubit zero, because according to the algorithm described in Section 3.2, we interchange the nonlocal qubit with the local qubit zero. On the IBM Regatta p690+, the CPU time to carry out  $H_n$  on a local qubit  $n$  can differ by a factor of three depending on the qubit number  $n$ . This is a result of how the local memory addresses are accessed and is thus due to the architecture of the machine. Techniques to speed up the

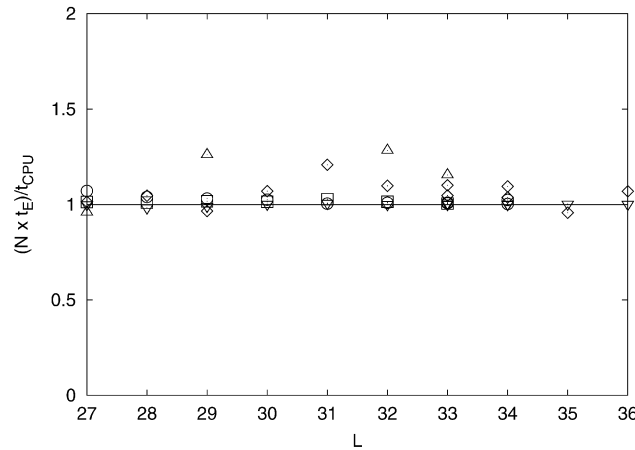


Fig. 5. Elapsed time  $t_E$  divided by the CPU time  $t_{CPU}$  and multiplied by the number of MPI processes  $N$  as a function of the number of qubits  $L$ . Diamonds: IBM Regatta p690+; squares: SGI Altix 3700; flipped triangles: IBM Blue Gene/L; circles: Hitachi SR11000/J1; triangles: Cray X1E. The horizontal line at  $N \times t_E / t_{CPU} = 1$  shows the ideal case in which the exponential increase of the problem size is exactly compensated by the exponential increase in the number of processors and memory size. Deviations from the ideal case are due to interprocessor communication.

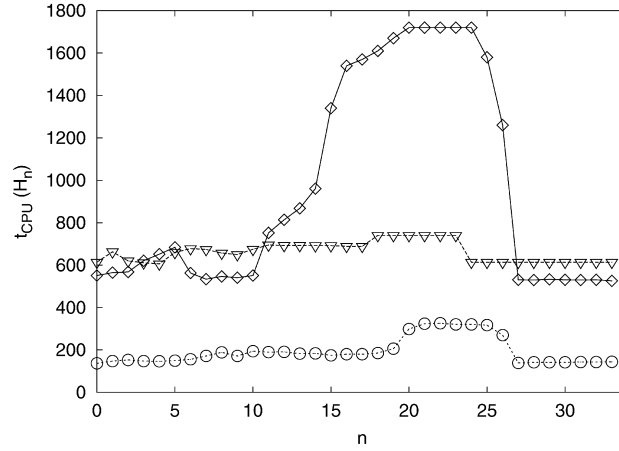


Fig. 6. CPU time  $t_{\text{CPU}}$  for a Hadamard operation  $H_n$  carried out on qubit  $n$  of a quantum computer with  $L = 34$  qubits ( $0 < n \leq 33$ ). Diamonds: IBM Regatta p690+; flipped triangles: IBM Blue Gene/L; circles: Hitachi SR11000/J1. The lines are guides to the eye. For  $15 \leq n \leq 26$ , the increase of CPU time on the IBM Regatta p690+ is due to less appropriate memory access and can be reduced by appropriate optimization techniques [4].

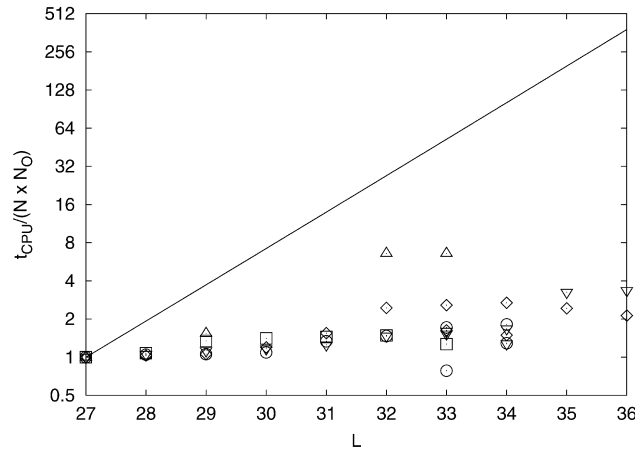


Fig. 7. CPU time  $t_{\text{CPU}}$  divided by the number of quantum operations  $N_O$  multiplied by the number of MPI processes  $N$  as a function of the number of qubits  $L$ . Diamonds: IBM Regatta p690+; squares: SGI Altix 3700; flipped triangles: IBM Blue Gene/L; circles: Hitachi SR11000/J1; triangles: Cray X1E. The line is given by  $27 \times 2^{L-27} / N_O$ .

memory access will be discussed in a future application [4]. On the IBM Blue Gene/L and on the Hitachi SR11000/J1 we only see a relatively small increase in CPU time if we operate  $H_n$  on qubits with increasing number  $n$ . The difference in behavior between the IBM Regatta p690+ and the two other machines is also reflected in Fig. 5, although there it is much less clear. Fig. 6 clearly shows that the Hitachi SR11000/J1 is significantly faster than the IBM Regatta p690+ and the IBM Blue Gene/L.

Fig. 7 shows  $t_{\text{CPU}} / (N \times N_O)$  as a function of  $L$ ,  $N_O$  being the number of quantum operations (not including swap commands). All data listed in Tables 5–9 is processed. On all machines, the number of MPI processes  $N = 2^{L-27}$ , except on the IBM Blue Gene/L,  $N = 8 \times 2^{L-27}$ . Therefore, for a program that parallelizes 100% we expect that  $t_{\text{CPU}} = \alpha N_O 2^{L-27}$ , where  $\alpha$  is a constant. Hence, in the ideal case  $t_{\text{CPU}} / (N \times N_O)$  should be equal to  $\alpha$ . From Fig. 7 it can be seen that for all machines,  $t_{\text{CPU}} / (N \times N_O)$  is slightly increasing as a function of  $L$ . The deviation from the ideal performance is due to the communication between various MPI processes. The corresponding increase in CPU time seems to be the largest for the IBM Regatta p690+ and the Cray X1E. However, the overall scaling properties of the simulation software are extremely good. For comparison we depict the line, given by  $27 \times 2^{L-27} / N_O$ , in Fig. 7. This line expresses the expected behavior of  $t_{\text{CPU}} / (N \times N_O)$  as a function of  $L$  on a non-parallel computer, that is a computer for which  $N = 1$ . Clearly, with the available hardware and the present simulation software, it is possible to beat the exponential growth of the simulation problem by simply increasing the memory and the number of CPUs by a factor of two for each qubit added, at least up to 36 qubits.

From Tables 5–9 it is clear that from the point of view of the user, that is in terms of elapsed time, the IBM Blue Gene/L is the fastest computer. If we make a comparison based on the CPU time then the Cray X1E is the fastest machine, followed by the Hitachi SR11000/J1. We emphasize that, except for the Cray X1E, we have used the same source code on all machines, that is we did not make an effort to adapt the code for a particular machine.

Out of curiosity and to demonstrate the portability of the parallel simulation code, we also test the software on a cluster of three IBM Thinkpad T43P notebooks and one IBM Think Centre A50P PC, using MPICH2 under Windows XP. Communication between the machines is handled by a 100 Mbit Ethernet US Robotics router. In Table 10 we show the performance results for carrying out a Hadamard operation on each qubit. Explicit measurement of the time spent for communication between the machines shows that for the example with  $L = 27$  qubits, about half of  $t_E$  and  $t_{CPU}$  is spent on network communication.

## 7. Discussion

The parallel quantum computer simulator presented in this paper is an application that shows nearly perfect scaling with the number of CPUs and problem size. It is a demanding application in that it can use a significant part of the available memory and CPUs and can put a heavy burden on the communication network. Therefore, this simulator may find application as a new type of benchmark to assess the computational power of the new generation of high-end computer systems.

## Acknowledgements

Support from the “Nederlandse Stichting Nationale Computer Faciliteiten (NCF)” is gratefully acknowledged. We thank Gyan Bhanot (IBM Yorktown) for helping us with porting the code to the IBM BlueGene/L, ASTRON (NL) for providing access to the IBM BlueGene/L and Arnold Meijsters (RuG) for helping us with running the simulations on the IBM BlueGene/L. We are grateful to J. Thorbecke (Cray) for performing the benchmarks on the Cray X1E. We thank W. Frings at FZJ for fruitful discussions on hardware specifics of the IBM p690+. This work is partially supported by the Japanese Ministry of Internal Affairs and Communications (Soumu-sho). Finally, we thank the Supercomputer Center, Institute for Solid State Physics, University of Tokyo for the facilities and the use of Hitachi SR11000/J1.

## Appendix A. Quantum Computer Circuit Editor (QCCE)

In this appendix we discuss the text-formatted file, generated by QCCE, that is used as input for the simulator if it has to simulate a Hadamard operation performed on each qubit of a 32-qubit quantum computer. Lines starting with “!” or “#” are considered as comments. We use these symbols to add comments to the following text generated by QCCE:

```

QUBITS 32                ! The command QUBITS sets the size of the quantum computer
INITIAL STATE 0          ! The initial state of the quantum computer is set to |0...0>
MPIPROCESSES 32          ! Number of MPI processes is set to 32
                          ! (not used by the OpenMP code)
H 0                      ! Hadamard operation on qubit 0.
.                        ! These lines are omitted here but contain commands for.
.                        ! Hadamard operations carried out on qubits 1-25.
.
H 26                    ! Hadamard operation on qubit 26
SWAP 1 0 27              ! Command to swap 1 pair of qubits: qubits 0 and 27
H 27                    ! Hadamard operation on qubit 27
SWAP 1 27 28             ! Command to swap qubits 27 and 28
H 28                    ! Hadamard operation on qubit 28
SWAP 1 28 29             ! Command to swap qubits 28 and 29
H 29                    ! Hadamard operation on qubit 29
SWAP 1 29 30             ! Command to swap qubits 29 and 30
H 30                    ! Hadamard operation on qubit 30
SWAP 1 30 31             ! Command to swap qubits 30 and 31
H 31                    ! Hadamard operation on qubit 31
BEGIN MEASUREMENT
DO MEASUREMENT 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 31
                          ! Measurement operation on the listed qubits
SWAP 5 31 1 2 3 4 0 27 28 29 30 ! Command to swap 5 pairs of qubits: (31,0), (1,27), (2,28),
                          ! (3,29), and (4,30)
DO MEASUREMENT 0 27 28 29 30 ! Measurement operation on the listed qubits
END MEASUREMENT

```

As can be seen from the example, the general format of a command consists of a keyword followed by some values. Note that the swap command is not the same as the quantum swap operation. The swap command carries out the qubit interchange described in Section 3.2.



## Appendix B. Simulation results

Table 5

Performance results for the IBM Regatta p690+ for the simulation of various quantum algorithms on a  $L$ -qubit ideal quantum computer

| $L$ | $N$ | Memory [GB] | $N_O$ | $t_E$ [s] | $t_{CPU}$ [s] | Quantum algorithm      |
|-----|-----|-------------|-------|-----------|---------------|------------------------|
| 27  | 1   | 2           | 27    | 144       | 142           | H                      |
| 28  | 2   | 4           | 28    | 159       | 304           | H                      |
| 29  | 4   | 8           | 29    | 156       | 646           | H                      |
| 30  | 8   | 16          | 30    | 202       | 1510          | H                      |
| 31  | 16  | 32          | 31    | 3605      | 4040          | H                      |
| 32  | 32  | 64          | 32    | 453       | 13200         | H                      |
| 33  | 64  | 128         | 33    | 492       | 28600         | H                      |
| 34  | 128 | 256         | 34    | 526       | 61500         | H                      |
| 33  | 64  | 128         | 286   | 2568      | 157000        | 3-11QBA: 292+585+1170  |
| 34  | 128 | 256         | 493   | 4019      | 496000        | 2-17QBA: 26214+104857  |
| 35  | 256 | 512         | 194   | 2374      | 635000        | 5-7QBA: 7+9+19+35+65   |
| 36  | 512 | 1024        | 342   | 4095      | 1960000       | 3-12QBA: 781+1054+3296 |

$N$ : Number of MPI processes; Memory: Memory required to store  $|\Phi\rangle$ ;  $N_O$ : Number of quantum operations (not including swap commands);  $t_E$ : Elapsed time;  $t_{CPU}$ : CPU time; H: Hadamard operation;  $n$ - $m$ QBA:  $\sum_{i=1}^n x_i$ : Qubit adder to add the content of  $n$   $m$ -qubit registers. The  $n$  registers contain each one number  $x$  represented in binary notation by  $m$  bits. The CPU time and the elapsed time include the time for measuring each qubit.

Table 6

Same as Table 5 for the SGI Altix 3700 system

| $L$ | $N$ | Memory [GB] | $N_O$ | $t_E$ [s] | $t_{CPU}$ [s] | Quantum algorithm     |
|-----|-----|-------------|-------|-----------|---------------|-----------------------|
| 27  | 1   | 2           | 27    | 308       | 304           | H                     |
| 28  | 2   | 4           | 28    | 343       | 679           | H                     |
| 29  | 4   | 8           | 29    | 446       | 1754          | H                     |
| 30  | 8   | 16          | 30    | 483       | 3809          | H                     |
| 31  | 16  | 32          | 31    | 518       | 8063          | H                     |
| 32  | 32  | 64          | 32    | 543       | 17127         | H                     |
| 33  | 64  | 128         | 286   | 4112      | 216772        | 3-11QBA: 292+585+1170 |

Table 7

Same as Table 5 for the IBM Blue Gene/L

| $L$ | $N$  | Memory [GB] | $N_O$ | $t_E$ [s] | $t_{CPU}$ [s] | Quantum algorithm      |
|-----|------|-------------|-------|-----------|---------------|------------------------|
| 27  | 8    | 2           | 27    | 40        | 320           | H                      |
| 28  | 16   | 4           | 28    | 43        | 700           | H                      |
| 29  | 32   | 8           | 29    | 48        | 1550          | H                      |
| 30  | 64   | 16          | 30    | 52        | 3320          | H                      |
| 31  | 128  | 32          | 31    | 58        | 7370          | H                      |
| 32  | 256  | 64          | 32    | 70        | 17900         | H                      |
| 33  | 512  | 128         | 33    | 79        | 40100         | H                      |
| 34  | 1024 | 256         | 34    | 85        | 86800         | H                      |
| 33  | 512  | 128         | 286   | 649       | 332000        | 3-11QBA: 292+585+1170  |
| 34  | 1024 | 256         | 493   | 934       | 956000        | 2-17QBA: 26214+104857  |
| 35  | 2048 | 512         | 194   | 940       | 1920000       | 5-7QBA: 7+9+19+35+65   |
| 36  | 4096 | 1024        | 342   | 1707      | 6980000       | 3-12QBA: 781+1054+3296 |

Table 8

Same as Table 5 for the Hitachi SR11000/J1

| $L$ | $N$ | Memory [GB] | $N_O$ | $t_E$ [s] | $t_{CPU}$ [s] | Quantum algorithm     |
|-----|-----|-------------|-------|-----------|---------------|-----------------------|
| 27  | 1   | 2           | 27    | 75        | 70            | H                     |
| 28  | 2   | 4           | 28    | 79        | 152           | H                     |
| 29  | 4   | 8           | 29    | 82        | 318           | H                     |
| 30  | 8   | 16          | 30    | 87        | 679           | H                     |
| 31  | 16  | 32          | 31    | 108       | 1720          | H                     |
| 32  | 32  | 64          | 32    | 123       | 3900          | H                     |
| 33  | 64  | 128         | 33    | 148       | 9410          | H                     |
| 34  | 128 | 256         | 34    | 164       | 20500         | H                     |
| 33  | 128 | 128         | 286   | 586       | 74300         | 3-11QBA: 292+585+1170 |
| 34  | 128 | 256         | 493   | 1653      | 211000        | 2-17QBA: 26214+104857 |

Table 9  
Same as Table 5 for the Cray X1E

| $L$ | $N$ | Memory [GB] | $N_O$ | $t_E$ [s] | $t_{CPU}$ [s] | Quantum algorithm     |
|-----|-----|-------------|-------|-----------|---------------|-----------------------|
| 27  | 4   | 2           | 27    | 12        | 50            | H                     |
| 29  | 8   | 8           | 29    | 26        | 165           | H                     |
| 32  | 32  | 64          | 32    | 125       | 3117          | H                     |
| 33  | 64  | 128         | 286   | 1006      | 55750         | 3-11QBA: 292+585+1170 |

Table 10  
Same as Table 5 for a cluster of three IBM Thinkpad T43P notebooks (each having 1 GB of memory and a 2.13 GHz CPU) and one IBM Think Centre A50P PC (having 2 GB of memory and a 2.8 GHz CPU), using MPICH2 under Windows XP

| $L$ | $N$ | Memory [GB] | $N_O$ | $t_E$ [s] | $t_{CPU}$ [s] | Quantum algorithm |
|-----|-----|-------------|-------|-----------|---------------|-------------------|
| 26  | 1   | 1           | 26    | 73        | 73            | H                 |
| 26  | 4   | 1           | 26    | 81        | 290           | H                 |
| 27  | 4   | 2           | 27    | 304       | 1220          | H                 |

Communication between the machines is handled by a 100 Mbit Ethernet US Robotics router. Explicit measurement of the time spent for communication between the machines shows that for the example with  $L = 27$  qubits half of  $t_E$  and  $t_{CPU}$  is spent on the swap operation.

## References

- [1] M. Nielsen, I. Chuang, Quantum Computation and Quantum Information, Cambridge University Press, Cambridge, 2000.
- [2] <http://www.qc.fraunhofer.de>.
- [3] H. De Raedt, K. Michielsen, in: M. Rieth, W. Schommers (Eds.), Handbook of Theoretical and Computational Nanotechnology, American Scientific Publishers, Los Angeles, 2006.
- [4] B. Trieu, et al., Preprint.
- [5] H. De Raedt, A.H. Hams, K. Michielsen, K. De Raedt, Comput. Phys. Comm. 132 (2000) 1.
- [6] QCE can be downloaded from <http://www.compphys.org/qce.htm>.
- [7] H. De Raedt, K. Michielsen, A. Hams, S. Miyashita, K. Saito, Eur. Phys. J. B 27 (2002) 15.
- [8] Message Passing Interface Forum, <http://www.mpi-forum.org> (URL last accessed on February 2, 2006).
- [9] L.I. Schiff, Quantum Mechanics, McGraw-Hill, New York, 1968.
- [10] G. Baym, Lectures on Quantum Mechanics, W.A. Benjamin, Reading MA, 1974.
- [11] L.E. Ballentine, Quantum Mechanics: A Modern Development, World Scientific, Singapore, 2003.
- [12] D.P. DiVincenzo, Phys. Rev. A 51 (1995) 1015.
- [13] A. Barenco, C.H. Bennett, R. Cleve, D.P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J.A. Smolin, H. Weinfurter, Phys. Rev. A 52 (1995) 3457.
- [14] S. Bettelli, Ph.D. Thesis, University of Trento, 2002.
- [15] P.W. Shor, SIAM Rev. 41 (1999) 303.
- [16] A. Ekert, R. Jozsa, Rev. Mod. Phys. 68 (1996) 733.
- [17] G.H. Hardy, E.M. Wright, An Introduction to the Theory of Numbers, Oxford University Press, Oxford, 2000.